

Регулярные выражения в C#

Регулярные выражения в C#

Регулярные выражения в C# — это мини-язык для описания шаблонов текста, который реализован через класс `Regex` в .NET. Они позволяют искать, проверять, извлекать и изменять строки по гибким правилам, которые трудно или громоздко реализовать обычными циклами и `IndexOf`.

Где и как используются `Regex` в C#

В .NET центральная точка работы с регулярками — класс **`System.Text.RegularExpressions.Regex`**. Он умеет проверять, есть ли совпадение, искать первое или все совпадения, заменять части строки и разбивать строку по сложному разделителю.

Основные методы Regex

Regex.IsMatch(string input, string pattern) — есть ли совпадение с шаблоном.

Regex.Match(string input, string pattern) — первое совпадение с подробной информацией (значение, индекс, группы).

Regex.Matches(string input, string pattern) — все совпадения (коллекция MatchCollection).

Regex.Replace(string input, string pattern, string replacement) — замена по шаблону.

Regex.Split(string input, string pattern) — разбить строку по шаблону-разделителю.

Regex.IsMatch

`Regex.IsMatch(string input, string pattern)` — есть ли совпадение с шаблоном.

```
string input = Console.ReadLine();
string pattern = "abc";
bool isContains = Regex.IsMatch(input, pattern);
if (isContains) {
    Console.WriteLine("В строке input есть \"abc\"");
}
else
{
    Console.WriteLine("В строке input нет \"abc\"");
}
```

```
ab888c
В строке input нет "abc"
```

```
8888
В строке input нет "abc"
```

```
abc88
В строке input есть "abc"
```

Regex.Match

```
string input = Console.ReadLine();
string pattern = "abc";
// ищет первое вхождение
Match resMatch = Regex.Match(input, pattern);
// Success - bool показывает, нашлось ли совпадение
if (resMatch.Success) {
    Console.WriteLine("В строке input есть \"abc\"");
    // возвращает выделенную подстроку
    Console.WriteLine(resMatch.Value);
    // возвращает позицию в исходной строке input
    Console.WriteLine(resMatch.Index);
    // возвращает имя захваченной группы( в нашем примере нет групп вернёт 0)
    Console.WriteLine(resMatch.Name);
}
```

Выбрать Консоль отладки Microsoft Visual Studio

```
0123abc88abc
В строке input есть "abc"
abc
4
0
```

Regex.Match(string input, string pattern) — первое совпадение с подробной информацией (значение, индекс, группы).

Regex.Matches

Regex.Matches(string input, string pattern) — все совпадения
(коллекция MatchCollection).

```
string input = Console.ReadLine();
string pattern = "abc";
// ищет первое вхождение
MatchCollection resMatches = Regex.Matches(input, pattern);
// Count - количество совпадений
if (resMatches.Count > 0) {
    Console.WriteLine($"В строке input есть \"abc\": {resMatches.Count} вхождений");
    foreach (Match m in resMatches)
    {
        Console.Write(m.Value + " ");
        Console.Write(m.Index + " ");
        Console.WriteLine(m.Name);
        Console.WriteLine();
    }
}
```

Консоль отладки Microsoft Visual Studio

```
0123abc 77 abcabc
В строке input есть "abc": 3 вхождений
abc 4 0
abc 11 0
abc 14 0
```

```
C:\Users\nikka\source\repos\ConsoleApp3\ConsoleApp3\bin\Debug\
(0x0).
```

Чтобы автоматически закрывать консоль при остановке отладки

Regex.Replace

Regex.Replace(string input, string pattern, string replacement) — замена по шаблону.

```
string input = Console.ReadLine();  
string pattern = "abc";  
// в строке input заменяем "abc" на "777"  
string res = Regex.Replace(input, pattern, "777");  
Console.WriteLine($"Изменёная строка: {res}");
```

 Консоль отладки Microsoft Visual Studio

```
String abc replace  
Изменёная строка: String 777 replace
```

Regex.Split

Regex.Split(string input, string pattern) — разбить строку по шаблону-разделителю.

```
string input = Console.ReadLine();  
string pattern = "abc";  
// строку input разделяем по шаблону "abc"  
string[] res = Regex.Split(input, pattern);  
foreach (string s in res)  
{  
    Console.WriteLine(s);  
}
```

CA Консоль отладки Microsoft Visual Studio

```
string abc line2 two abc line3 string abc  
string  
    line2 two  
    line3 string
```


Синтаксис. Базовые символы

Шаблон регулярного выражения строится из простых элементов (символов и команд).

Базовые символы и экранирование:

- Обычные символы (буквы, цифры) совпадают сами с собой: шаблон `abc` ищет «`abc`».
- Спецсимволы: `.` `^` `$` `*` `+` `?` `{` `}` `[]` `\` `|` `()` — их нужно экранировать `\`, если хотите воспринимать как обычный символ, например `\.` для точки.

```
string input = Console.ReadLine();
string pattern = @"\$";
bool isContains = Regex.IsMatch(input, pattern);
if (isContains) {
    Console.WriteLine("В строке input есть \"$\");
}
```

Консоль отладки Microsoft Visual Studio

```
string with $
В строке input есть "$"
```

Синтаксис. Классы символов

Классы символов (набор допустимых символов):

- [abc] — любой из a, b или c.
- [0-9] — любая цифра от 0 до 9.
- [A-Za-z] — латинская буква.
- [^0-9] — любой символ, кроме цифры (^ внутри [] — отрицание).

```
string input = Console.ReadLine();
string pattern = "[abc]";
bool isContains = Regex.IsMatch(input, pattern);
if (isContains) {
    Console.WriteLine("В строке input есть символы \"a\" или \"b\" или \"c\"");
}
```

Консоль отладки Microsoft Visual Studio

```
string with simbol a
В строке input есть символы "a" или "b" или "c"
```

```
string input = Console.ReadLine();
string pattern = "[0-9]";
bool isContains = Regex.IsMatch(input, pattern);
if (isContains) {
    Console.WriteLine("В строке input есть символы от 0 до 9");
}
```

 Консоль отладки Microsoft Visual Studio

```
string with 777 000
В строке input есть символы от 0 до 9
```

```
string input = Console.ReadLine();
string pattern = "[^0-9]";
bool isContains = Regex.IsMatch(input, pattern);
if (isContains) {
    Console.WriteLine("В строке input нет цифр");
}
```

 Консоль отладки Microsoft Visual Studio

```
just string
В строке input нет цифр
```

```
string input = Console.ReadLine();
string pattern = "[^a-z]";
bool isContains = Regex.IsMatch(input, pattern);
if (isContains) {
    Console.WriteLine("В строке input нет букв");
}
```

 Консоль отладки Microsoft Visual Studio

```
999
В строке input нет букв
```

```
string input = Console.ReadLine();  
string pattern = "[A-Za-z]";  
bool isContains = Regex.IsMatch(input, pattern);  
if (isContains) {  
    Console.WriteLine("В строке input есть буквы латинского алфавита");  
}
```

Консоль отладки Microsoft Visual Studio

```
string tttt  
В строке input есть буквы латинского алфавита
```

```
string input = Console.ReadLine();  
string pattern = "[А-Яа-я]";  
bool isContains = Regex.IsMatch(input, pattern);  
if (isContains) {  
    Console.WriteLine("В строке input есть буквы кириллицы");  
}
```

Консоль отладки Microsoft Visual Studio

```
всем пока  
В строке input есть буквы кириллицы
```

Сокращённые классы символов

\d — цифра (0–9).

\D — не цифра.

\w — «словесный» символ: буква, цифра или _.

\W — не «словесный» символ.

\s — пробельный символ (пробел, таб, перевод строки и т.д.).

\S — не пробельный символ.

Якоря (позиции в строке, а не сами символы)

`^` — начало строки (или строки в многострочном режиме).

`$` — конец строки.

`\b` — граница слова (переход между `\w` и `\W`).

`\B` — не граница слова.

```
// \d - одна цифра ; \s - пробельный символ
// \w - буква
string input = Console.ReadLine();
string pattern = @"\d\s\w";
Match res = Regex.Match(input, pattern);
if (res.Success) {
    Console.WriteLine(res.Value);
}
```

CA Консоль отладки Microsoft Visual Studio

```
string 6 simbol
6 s
```

```
// \W - не буква ; \s - пробельный символ
// \D - не цифра
string input = Console.ReadLine();
string pattern = @"\W\s\D";
Match res = Regex.Match(input, pattern);
if (res.Success) {
    Console.WriteLine(res.Value);
}
```

CA Выбрать Консоль отладки Microsoft Visual Studio

```
string # rr
# r
```

```
// ^ - начало строки ; \w - буква
// \d - цифра ; \s - пробел ; $ - строка заканчивается
string input = Console.ReadLine();
string pattern = @"^\d\d\w\s\w$";
Match res = Regex.Match(input, pattern);
if (res.Success) {
    Console.WriteLine(res.Value);
}
else
{
    Console.WriteLine("Строка не подходит под формат");
}
```

```
12a ш
12a ш
```

```
12a 888
Строка не подходит под формат
```

```
// ^ - начало строки ;
// \d - цифра ;
string input = Console.ReadLine();
string pattern = @"^\d\d";
Match res = Regex.Match(input, pattern);
if (res.Success) {
    Console.WriteLine(res.Value);
}
else
{
    Console.WriteLine("Строка не подходит под формат");
}
```

```
33 строка
33
```

```
4к фул HD
Строка не подходит под формат
```


Квантификаторы: сколько раз повторить

Основные квантификаторы:

* — 0 или более повторений (может не быть вообще).

+ — 1 или более повторений (минимум один).

? — 0 или 1 раз (опциональный элемент).

{n} — ровно n раз.

{n,} — от n до бесконечности раз.

{n,m} — от n до m раз включительно.

```
// \d* – должно быть 0 или больше цифр
// затем три символа &
// затем \d+ один или более цифр
string input = Console.ReadLine();
string pattern = @"^d*&&&d+";
Match res = Regex.Match(input, pattern);
if (res.Success) {
    Console.WriteLine(res.Value);
}
else
{
    Console.WriteLine("Строка не подходит под формат");
}
```

```
t&&&333
&&&333
```

```
777&&&0
777&&&0
```

```
rrr&&&rrr
Строка не подходит под формат
```

```
// Пример: шаблон ^\d{4}-\d{2}-\d{2}$
// описывает строку вида год-месяц-день
// (например, 2023-02-21) и запрещает лишние символы слева/справа.
string input = Console.ReadLine();
string pattern = @"^\d{4}-\d{2}-\d{2}$";
Match res = Regex.Match(input, pattern);
if (res.Success) {
    Console.WriteLine(res.Value);
}
```

Консоль отладки Microsoft Visual Studio

```
2023-02-01
2023-02-01
```

Жадные и «ленивые» квантификаторы:

По умолчанию *, +, {n,m} — жадные: захватывают как можно больше, оставляя шаблон валидным.

Добавление ? делает их «ленивыми» (нежадными): *?, +?, {n,m}? пытаются взять минимум, при котором совпадение возможно.

```
// \d*? — ищет сразу цифры с ---  
// \w? — ищет один символ  
string input = Console.ReadLine();  
string pattern = @"\d*?---\w?";  
Match res = Regex.Match(input, pattern);  
if (res.Success) {  
    Console.WriteLine(res.Value);  
}
```

Строка:

123---a

Оба шаблона (\d*---\w? и \d*?---\w?) найдут 123---a.

Строка:

9999xyz123---bzzz

Оба найдут 123---b (цифры перед --- плюс --- плюс b).

Жадный \d* старается схватить как можно больше цифр, но если после этого не удаётся найти ---, он откатывается назад, сокращая количество цифр, пока --- не совпадёт.

Ленивый \d*? наоборот сначала берёт 0 цифр, сразу пытается найти ---, если не получается, берёт 1 цифру, снова пробует ---, и так далее, пока не найдёт совпадение или не дойдёт до конца строки.

Группы, альтернативы и захват данных

Группы и альтернативы позволяют собирать сложные шаблоны из частей.

Скобочные группы:

(абв) — группа: позволяет применять квантификатор к целому фрагменту и извлекать его из совпадения.

Нумерованные группы: первая пара () — группа 1, затем группа 2 и т.д.

Захваченный текст можно получить через `Match.Groups[i].Value` в C#.

Пример: шаблон `(\d{4})-(\d{2})-(\d{2})` даёт три группы: год, месяц, день.

```
// (\d{4}) - первая группа
// (\d{2}) - вторая группа
// (\d{2}) - третья группа
string input = Console.ReadLine();
string pattern = @"(\d{4})-(\d{2})-(\d{2})";
Match res = Regex.Match(input, pattern);
if (res.Success) {
    foreach (Group el in res.Groups)
    {
        Console.WriteLine(el.Name + " " + el.Value);
    }
}
```

Консоль отладки Microsoft Visual Studio

```
date 2026-01-01
0 2026-01-01
1 2026
2 01
3 01
```

Альтернатива и «ИЛИ»

a|b — либо a, либо b.

кolor|цвет — совпадёт либо с «кolor», либо с «цвет».

Пример: (https?|ftp)://\S+ — протокол http или https или ftp, затем :// и непробельные символы.

```
// (цвет|color) – прописано либо то либо это
// затем пробел и #
// 6 цифр
string input = Console.ReadLine();
string pattern = @"(цвет|color) #\d{6}";
Match res = Regex.Match(input, pattern);
if (res.Success) {
    Console.WriteLine(res.Value);
}
else
{
    Console.WriteLine("Строка не подходит под формат");
}
```

```
небо цвет #888888
цвет #888888
```

```
травa color #777777
color #777777
```

```
цвет #3
Строка не подходит под формат
```

Пример задания

Проверить Email на корректность.

Базовая запись : «group24@mail.ru»

Разбор записи, у всех почт повторяется @ и .

До @ пишется любое количество букв или цифр.

После @ и перед . пишется название компании почты – только буквы.

После . пишется домен из 2 букв.

Задание

1. Найти все числа в строке

- Ввод: строка с текстом и числами, например: У меня 2 кошки и 15 рыбок.

2. Проверить, начинается ли строка с «abc»

- Ввод: строка с клавиатуры.

3. Проверить формат простого номера телефона

- Формат: плюс, ровно 3 цифры, пробел, 3 цифры, пробел, 4 цифры

4. Удалить все лишние пробелы между словами

- Ввод: строка с несколькими пробелами подряд, например: Привет мир C#.
- Задача: заменить все последовательности из нескольких пробелов на один пробел.

5. Найти все слова, начинающиеся с буквы «a» или «A»

- Ввод: строка на английском, например: Apple and avocado is fruits.
- Подсказка: [тут что то]\разные буквы